



RÉPUBLIQUE TUNISIENNE
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
Université de Carthage
**Institut National des Sciences Appliquées
et de Technologie**



RAPPORT DE PROJET EN DATA MINING

Spécialité : Génie Logiciel
Niveau : 4^e année

Étude Comparative des Algorithmes de Machine Learning pour la Prévision du Succès des Films

Présenté par

BEN KALIA MOHAMED SAHBI

ENGAZZOU LEITH

DAHMOUL YASSINE

AYADI TAKOUA

Année Universitaire : 2025 – 2026

Table des matières

1	Introduction	4
1.1	Contexte et Motivation	4
1.2	Problématique	4
1.3	Objectifs du Projet	4
1.4	Structure du Rapport	4
2	Analyse Exploratoire des Données (EDA)	5
2.1	Source et Description du Jeu de Données	5
2.2	Audit de la Qualité des Données	5
2.3	Visualisations Clés et Hypothèses	6
2.3.1	Distribution de la Variable Cible (Revenue)	6
2.3.2	Corrélation Budget vs Revenu	6
2.3.3	Analyse du Casting (<code>crew</code>)	7
2.4	Statistiques Descriptives	8
3	Feature Engineering : Ingénierie des Variables	9
3.1	Création de nouvelles variables	9
3.2	Suppression de variables non pertinentes	10
4	Prétraitement des Données (Preprocessing)	12
4.1	Suppression des doublons	12
4.2	Correction des valeurs de pays (Country)	12
4.3	Encodage des Variables Catégorielles et Réduction de Dimensionnalité	12
4.3.1	Pays de Production (<code>country</code>) : Regroupement Stratégique	12
4.3.2	Genres Cinématographiques (<code>genres</code>) : Encodage Multi-Hot Exhaustif	13
4.3.3	Langue Originale (<code>original_language</code>) : Sélection du Top 10	14
5	Modélisation et Étude Comparative	16
5.1	Modèle K-Nearest Neighbors (KNN)	16
5.1.1	Principe du KNN pour la régression	16
5.1.2	Implémentation et code commenté	16
5.1.3	Chargement et préparation des données	16
5.1.4	Standardisation des caractéristiques	17
5.1.5	Recherche d'hyperparamètres par GridSearchCV	17
5.1.6	Évaluation et métriques	17
5.1.7	Résultats et analyse	17
5.2	Modèle Arbre de Décision (Decision Tree)	18
5.2.1	Principe de l'arbre de régression	18
5.2.2	Implémentation et code commenté	19
5.2.3	Grille d'hyperparamètres	19
5.2.4	Recherche et évaluation	19
5.2.5	Résultats et analyse	19
5.3	Modèle d'Ensemble (KNN + Decision Tree)	20
5.3.1	Principe de la combinaison	20
5.3.2	Implémentation et code commenté	20
5.3.3	Test de différentes stratégies de pondération	21
5.3.4	Évaluation finale	21
5.3.5	Résultats et analyse	21
5.4	Comparaison et discussion	22
5.4.1	Synthèse des performances	22
5.4.2	Interprétation des résultats	22
5.4.3	Limites et pistes d'amélioration	22
5.4.4	Synthèse des Premiers Résultats	23
5.5	Modèle Bagging Regression	23
5.5.1	Principe du Bagging	23
5.5.2	Implémentation et code commenté	23

5.5.3	Recherche d'hyperparamètres par GridSearchCV	23
5.5.4	Résultats et analyse	24
5.6	Modèle Forêt Aléatoire (Random Forest Regression)	25
5.6.1	Principe et Optimisation de la Diversité	25
5.6.2	Implémentation et Recherche d'Hyperparamètres	25
5.6.3	Résultats et Analyse des Performances	26
5.6.4	Analyse de l'Importance des Caractéristiques	26
5.6.5	Analyse Graphique des Résidus	27
5.7	Modèle Processus Gaussien (Gaussian Process Regression)	27
5.7.1	Principe du Processus Gaussien pour la régression	27
5.7.2	Choix du noyau et optimisation	28
5.7.3	Contraintes de calcul	28
5.7.4	Résultats et analyse des noyaux	28
5.7.5	Analyse de l'incertitude	29
5.7.6	Performances finales	30
6	Comparaison globale des modèles et discussion	30
6.1	Synthèse des performances	30
6.2	Analyse comparative	30
6.3	Limites et perspectives	31
7	Conclusion générale	31

1 Introduction

1.1 Contexte et Motivation

L'industrie cinématographique mondiale est devenue, au fil des décennies, l'un des secteurs économiques les plus lucratifs mais aussi l'un des plus risqués. Avec des budgets de production dépassant régulièrement les centaines de millions de dollars pour les blockbusters, la capacité à anticiper le succès commercial d'un film est devenue un enjeu stratégique majeur pour les studios de production et les distributeurs.

Traditionnellement, le succès d'un film reposait sur l'intuition des producteurs ou sur des campagnes marketing massives. Cependant, avec l'explosion de la donnée numérique et l'évolution des comportements des spectateurs, le **Data Mining** et le **Machine Learning** offrent aujourd'hui des outils analytiques permettant d'extraire des tendances invisibles à l'œil nu et de transformer l'incertitude en probabilité calculée.

1.2 Problématique

Le cœur de ce projet s'articule autour de l'énoncé suivant : « *Le Data Mining pour la prévision de succès des films* ».

L'enjeu est de déterminer s'il est possible, à partir des seules données disponibles avant la sortie d'un film (budget, casting, genre, etc.), d'estimer avec fiabilité les revenus qu'il générera. Il s'agit de transformer des données brutes en une connaissance prédictive capable d'orienter les décisions d'investissement.

1.3 Objectifs du Projet

Pour répondre à cette problématique, nos travaux se sont concentrés sur trois axes majeurs :

- **Analyse et préparation des données** : Explorer et transformer les données collectées pour en extraire les caractéristiques les plus influentes.
- **Étude comparative** : Implémenter et comparer les performances de plusieurs algorithmes de Machine Learning pour la régression.
- **Déploiement** : Concevoir une application permettant d'utiliser le modèle final pour réaliser des prédictions en temps réel.

1.4 Structure du Rapport

Le présent rapport est structuré comme suit : le premier chapitre est dédié à l'exploration et au prétraitement des données. Le deuxième chapitre détaille la phase de modélisation et la comparaison des algorithmes. Enfin, le dernier chapitre présente les résultats obtenus ainsi que l'application développée.

2 Analyse Exploratoire des Données (EDA)

Cette section présente l'analyse détaillée du jeu de données brut. L'objectif est de comprendre la structure des informations disponibles, d'identifier les anomalies nécessitant un nettoyage, et de valider les hypothèses qui guideront la phase de modélisation.

2.1 Source et Description du Jeu de Données

Le dataset utilisé dans le cadre de ce projet est extrait de la plateforme **Kaggle**, lui-même issu de la base de données **IMDB** (Internet Movie Database). Il constitue un échantillon représentatif de l'industrie cinématographique moderne.

Le fichier comporte **10 178 enregistrements** (lignes) et **12 variables** (colonnes). Le tableau ci-dessous détaille la sémantique et le type de chaque variable :

Colonne	Description	Type
names	Titre commercial du film	Texte
date_x	Date de sortie officielle	Date
score	Note moyenne des critiques/spectateurs (échelle 0-100)	Numérique
genre	Catégorie(s) cinématographique(s) (ex : Action, Drama)	Texte (Multi-valué)
overview	Synopsis narratif du film	Texte libre
crew	Liste brute du casting et des rôles	Texte semi-structuré
orig_title	Titre original (dans la langue de production)	Texte
status	État d'avancement (Released, Post Production, etc.)	Catégoriel
orig_lang	Code de la langue originale (ex : en, fr, ja)	Catégoriel
budget_x	Budget estimé de production (en USD)	Numérique
revenue	Revenus générés au Box-Office mondial (en USD)	Numérique
country	Pays d'origine de la production	Catégoriel

TABLE 1 – Dictionnaire des données du projet.

2.2 Audit de la Qualité des Données

Une inspection approfondie des variables a permis de mettre en évidence cinq défis majeurs qui nécessiteront des traitements spécifiques lors de la phase de nettoyage :

- **Structure complexe de la colonne crew** : Cette variable mélange les noms des acteurs et leurs rôles dans une chaîne de caractères unique. Cette structure non-atomique nécessite un *parsing* avancé pour isoler les entités "Acteurs" et construire notre métrique de "Star Power".
- **Multi-appartenance des genres** : Un film est rarement défini par un seul genre. La colonne **genre** contient des listes (ex : "*Science Fiction, Adventure, Action*"). Pour être exploitables, ces données devront subir un encodage de type *One-Hot Encoding*.
- **Anomalie et Encodage de la variable country** : Nous avons détecté une incohérence majeure dans la distribution géographique : une proportion anormalement élevée de films se voit attribuer le code "AU" (Australie), ce qui s'apparente à une valeur par défaut erronée dans le dataset source. Une étape de correction systématique est impérative pour rétablir les véritables pays de production (ex : US, UK, FR). Une fois corrigée, cette variable catégorielle subira un *One-Hot Encoding* afin de permettre au modèle d'évaluer l'impact du marché national d'origine sur le succès commercial mondial.
- **Encodage de la langue (orig_lang)** : La langue originale est une variable catégorielle nominale. Pour être interprétée par les modèles mathématiques (qui ne gèrent que des valeurs numériques), elle devra également subir une transformation, typiquement un *One-Hot Encoding*, en veillant à gérer la cardinalité pour ne pas exploser la dimensionnalité du dataset.

- **Filtrage du statut** : Le dataset contient des films non sortis en salle (statuts *In Production*, *Post Production*). Ces entrées, ne possédant pas de revenus définitifs, constituent du bruit pour notre variable cible et seront exclues.

2.3 Visualisations Clés et Hypothèses

2.3.1 Distribution de la Variable Cible (Revenue)

L'analyse de la variable cible **revenue** met en évidence une distribution **extrêmement asymétrique** (skewness positive forte).

- La majorité des films (le "ventre mou" de la distribution) génère des revenus modestes.
- Une minorité de "blockbusters" (valeurs extrêmes ou *outliers*) étire la distribution vers la droite. À titre d'exemple, *Avatar : The Way of Water* atteint plus de 2,3 milliards de dollars.

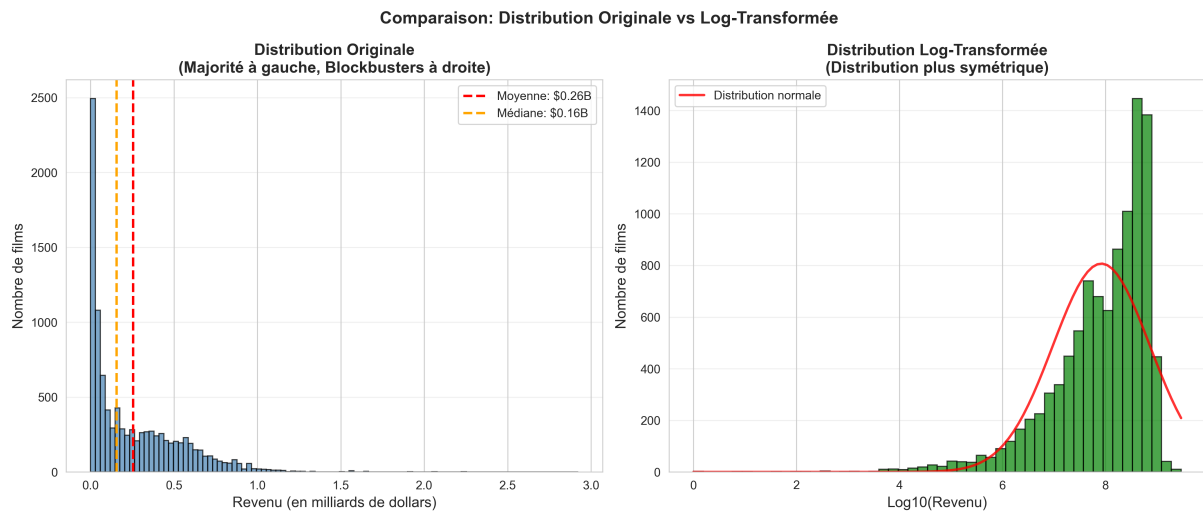


FIGURE 1 – Distribution des revenus (échelle linéaire vs logarithmique).

Implication pour la modélisation : Cette asymétrie justifie l'application d'une transformation logarithmique ($\log(1 + x)$) sur la variable cible. Cela permettra de normaliser la distribution, de réduire l'impact des outliers et d'améliorer la convergence des algorithmes de régression linéaire.

2.3.2 Corrélation Budget vs Revenu

L'intuition économique suggère qu'un budget élevé favorise un revenu élevé. Les données confirment cette hypothèse avec une **corrélation positive forte** (Coefficient de Pearson $r \approx 0.69$).

Cependant, la relation n'est pas strictement linéaire. On observe une hétéroscondasticité : la variance des revenus augmente avec le budget. Certains films à gros budget (ex : *Avatar*, 460M\$) performant exceptionnellement bien (2.3Md\$), tandis que d'autres échouent, illustrant la part de risque inhérente à l'industrie.

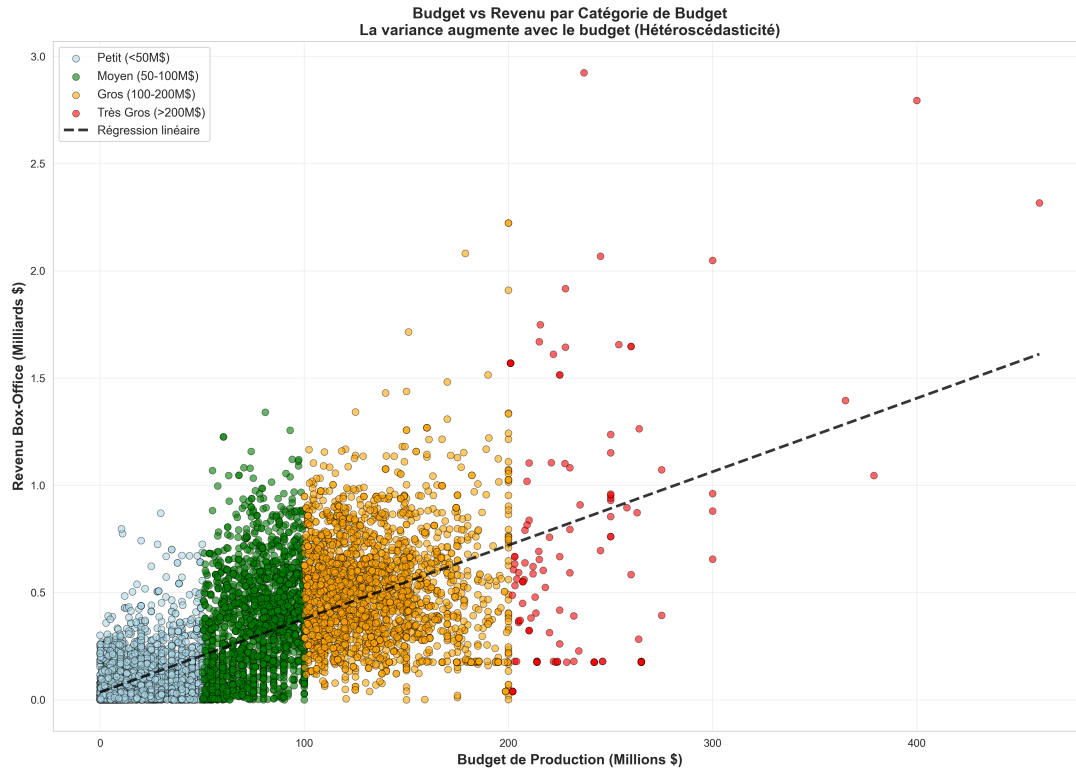


FIGURE 2 – Nuage de points : Budget de production vs Revenu Box-Office.

2.3.3 Analyse du Casting (crew)

L'analyse fréquentielle des acteurs révèle une distribution de type "loi de puissance". Une poignée d'acteurs "stars" participent à de nombreux films à succès, tandis que la grande majorité n'apparaît qu'une seule fois dans le dataset. Cette observation valide notre approche méthodologique future : l'utilisation d'une **moyenne bayésienne** pour le calcul du "Star Power". Une moyenne arithmétique simple serait trop sensible aux acteurs n'ayant fait qu'un seul film à succès par hasard.

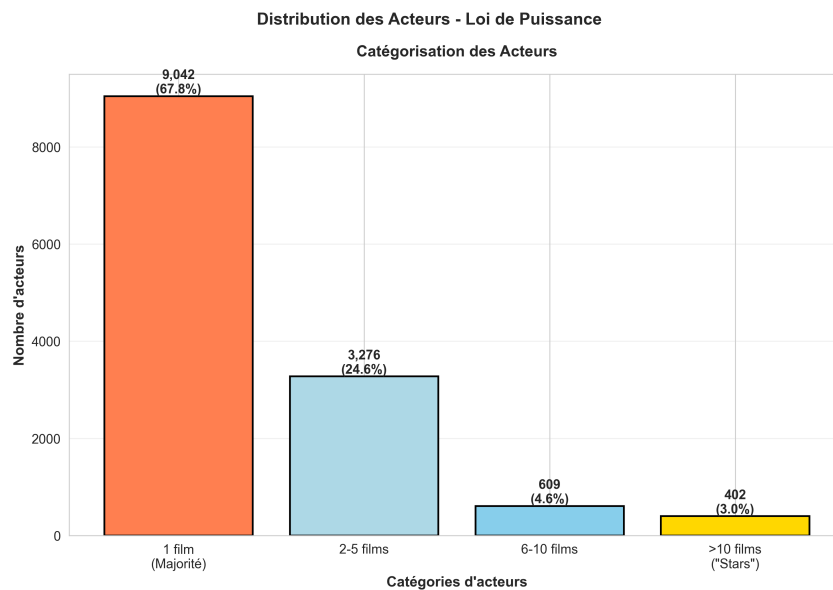


FIGURE 3 – Distribution des acteurs selon leur fréquence d'apparition.

2.4 Statistiques Descriptives

Le tableau suivant résume les statistiques fondamentales des variables numériques avant nettoyage :

Variable	Moyenne	Médiane	Écart-Type	Min / Max
Score	63.5	65.0	13.54	0 / 100
Budget (\$)	64.8M	50M	57M	0 / 460M
Revenu (\$)	253.1M	152.9M	277.7M	0 / 2.9Md

TABLE 2 – Statistiques descriptives (avant traitement des valeurs nulles).

Nous notons un nombre significatif de films affichant un budget ou un revenu nul. Ces valeurs, correspondant à des données manquantes ou non divulguées, feront l’objet d’un traitement spécifique (imputation ou suppression) dans la section suivante.

3 Feature Engineering : Ingénierie des Variables

La phase de Feature Engineering constitue une étape cruciale dans notre pipeline de modélisation, car elle vise à transformer, enrichir et sélectionner les variables afin d'améliorer la capacité prédictive de nos modèles. Cette étape repose sur une analyse approfondie de la pertinence métier de chaque variable, de sa disponibilité au moment de la prédiction, et de sa contribution statistique au modèle. Nous avons procédé à la fois à la création de nouvelles variables et à la suppression de variables jugées non pertinentes ou problématiques.

3.1 Création de nouvelles variables

Is_Sequel (Variable indicatrice de suite/franchise)

Nous avons créé une variable binaire nommée `Is_Sequel`, qui prend la valeur 1 si le film est une suite (sequel), un remake, un spin-off ou fait partie d'une franchise établie, et 0 sinon. Cette variable a été construite en analysant le titre du film et ses métadonnées associées, en recherchant notamment la présence de marqueurs typiques tels que des chiffres romains ou arabes dans le titre (ex. : « II », « III », « 2 », « 3 »), des mots-clés comme « Part », « Chapter », « Returns », « Revenge », ainsi qu'en croisant ces informations avec les données de franchises et de sagas fournies par l'API The Movie Database (TMDB), permettant d'identifier si un film appartient à une série ou à un univers cinématographique existant.

L'intuition derrière cette variable est fortement ancrée dans la réalité de l'industrie cinématographique : les suites et les films appartenant à des franchises bénéficient généralement d'une base de fans préexistante, d'une notoriété accumulée et d'un effort marketing réduit pour attirer le public, ce qui se traduit très souvent par des revenus d'ouverture et des recettes totales significativement plus élevés. En intégrant `Is_Sequel`, nous permettons au modèle de capturer cet effet de franchise qui constitue l'un des prédicteurs les plus puissants du succès commercial d'un film.

Star_Power (Pouvoir de la star)

Nous avons conçu une variable numérique complexe, le `Star_Power`, destinée à quantifier la "Bankability" du casting. Contrairement à une approche naïve qui consisterait à simplement sommer les revenus passés, nous avons développé une méthodologie en trois étapes pour garantir la robustesse statistique de cette métrique.

1. Problématique des moyennes simples Calculer la moyenne arithmétique des revenus des films précédents d'un acteur pose un problème de biais majeur. Un acteur ayant joué dans un seul film à grand succès (un "One-hit wonder") se retrouverait avec un score artificiellement égal à celui d'une superstar ayant une carrière longue et stable. Pour pallier ce problème de variance sur les petits échantillons, nous avons opté pour une approche de régularisation.

2. Calcul du Score Individuel (Moyenne Bayésienne) Pour chaque acteur présent dans le dataset, nous avons calculé un score pondéré inspiré de la formule utilisée par IMDB (Internet Movie Database) pour ses classements. Cette méthode, connue sous le nom de **Moyenne Bayésienne**, permet de "lisser" les scores des acteurs ayant peu de films vers la moyenne globale.

La formule appliquée est la suivante :

$$WR = \left(\frac{v}{v+m} \cdot R \right) + \left(\frac{m}{v+m} \cdot C \right) \quad (1)$$

Où :

- WR est le *Weighted Rating* (Score final de l'acteur).
- R est la moyenne arithmétique des revenus des films de l'acteur.
- v est le nombre de films dans lesquels l'acteur a joué (support).
- C est la moyenne globale des revenus de tous les acteurs du dataset.
- m est le seuil minimal de films requis (fixé de manière empirique, par exemple au 50^e percentile).

Justification du seuil $m = 5$: Après analyse de la distribution des filmographies, nous avons fixé le paramètre m à 5. Ce choix permet d'isoler statistiquement le "cœur" des acteurs professionnels (le Top 10% les plus actifs). L'effet de ce paramètre est double :

- Pour un **"One-Hit Wonder"** (acteur inconnu ayant joué dans un seul blockbuster, $v = 1$), le poids de la moyenne globale C est prépondérant. Son score est ainsi "écrasé" vers la normale, évitant une surestimation artificielle.
- Pour une **Star confirmée** (ex : Tom Cruise avec $v \approx 40$), le poids de m devient négligeable devant v . Le score final reflète alors quasi-intégralement sa performance personnelle R .

Cette formule garantit qu'un acteur avec peu de films aura un score proche de la moyenne globale C , tandis qu'un acteur avec une filmographie riche verra son score tendre vers sa performance réelle R .

3. Agrégation au niveau du Film (Pondération Top 3) Une fois le score individuel de chaque acteur calculé, nous avons déterminé le **Star_Power** global du film. Plutôt que de considérer l'ensemble du casting (qui inclut des figurants au score faible), nous nous sommes concentrés sur les **3 acteurs principaux** (Lead Roles), car ce sont eux qui figurent sur les affiches et attirent le public.

Nous avons appliqué une somme pondérée décroissante pour refléter la hiérarchie du casting :

$$Score_{Film} = (0.5 \times S_1) + (0.3 \times S_2) + (0.2 \times S_3) \quad (2)$$

Où S_1, S_2, S_3 sont les scores respectifs du 1^{er}, 2^{ème} et 3^{ème} acteur crédité. De plus, nous avons mis en place une stratégie d'imputation : si un acteur est inconnu ou nouveau dans la base, son score n'est pas fixé à zéro (ce qui pénaliserait injustement le film), mais remplacé par la moyenne globale C .

Cette approche rigoureuse permet à la variable **Star_Power** de capturer un signal fort et stable, corrélé à la réussite commerciale, sans introduire de bruit lié aux acteurs secondaires ou aux carrières naissantes.

Transformation de la variable date en Release_Year et Release_Month

La variable originale **date**, qui contenait la date de sortie complète du film au format texte (par exemple "2023-07-21"), n'était pas directement exploitable par nos algorithmes de machine learning sous sa forme brute. Nous avons donc procédé à sa décomposition en deux nouvelles variables numériques :

- **Release_Year (année de sortie)** : Cette variable capture les tendances temporelles et macro-économiques qui influencent les revenus du box-office. En effet, les recettes cinématographiques ont connu une inflation naturelle au fil des décennies due à l'augmentation du prix des billets, à l'expansion des marchés internationaux et à l'évolution des habitudes de consommation. L'année de sortie permet au modèle de contextualiser les performances d'un film dans son époque.
- **Release_Month (mois de sortie)** : Cette variable est particulièrement stratégique car elle capture l'effet de saisonnalité qui est fondamental dans l'industrie du cinéma. Il est bien documenté que certains mois de l'année sont associés à des recettes nettement supérieures : les mois d'été (juin, juillet, août) correspondent à la période des blockbusters estivaux, tandis que les mois de novembre et décembre bénéficient de la saison des fêtes et de la période des awards (course aux Oscars). À l'inverse, les mois de janvier et février sont traditionnellement considérés comme des périodes creuses (*dump months*), où les studios sortent des films à moindre potentiel commercial. En isolant le mois de sortie, nous permettons au modèle de capturer ces dynamiques saisonnières cruciales.

Après extraction de ces deux composantes, la colonne **date** originale a été supprimée du dataset puisque l'information qu'elle contenait est désormais intégralement représentée par les deux nouvelles variables.

3.2 Suppression de variables non pertinentes

Suppression de score (note moyenne du film)

La variable **score**, qui représente la note moyenne attribuée au film par les spectateurs et/ou les critiques (typiquement sur une échelle de 1 à 100), a été délibérément retirée du jeu de features. Bien que cette variable présente généralement une corrélation non négligeable avec les revenus, sa suppression est motivée par une raison fondamentale liée au réalisme du scénario de prédiction. En effet, le score d'un film n'est disponible qu'après sa sortie en salles, une fois que les spectateurs et les critiques ont eu l'occasion de le visionner et de le noter.

Or, l'objectif de notre modèle est de prédire les revenus d'un film avant ou au moment de sa sortie, c'est-à-dire à un stade où cette information n'existe tout simplement pas. Utiliser le score comme variable prédictive constituerait donc un cas typique de **fuite de données (data leakage)**, où une information

provenant du futur (postérieure à l'événement que l'on cherche à prédire) est injectée dans le modèle pendant l'entraînement. Ce phénomène conduit à une surestimation artificielle des performances du modèle lors de l'évaluation, mais engendre des performances médiocres en conditions réelles.

Suppression de overview (synopsis du film)

La variable `overview`, qui contient le résumé textuel ou le synopsis du film, a été supprimée du dataset. Cette décision repose sur plusieurs considérations. Premièrement, il s'agit d'une variable de type texte libre dont l'exploitation nécessiterait la mise en œuvre de techniques avancées de traitement du langage naturel (NLP), ce qui dépasse le cadre de notre étude actuelle centrée sur des approches de machine learning tabulaire. Deuxièmement, la qualité et la longueur des synopsis varient considérablement d'un film à l'autre, rendant l'extraction de features textuelles peu fiable. Troisièmement, l'information contenue dans le synopsis est en grande partie redondante avec le genre du film.

Suppression de original_name (titre original du film)

La variable `original_name`, correspondant au titre original du film, a été retirée des features. Le titre d'un film est un identifiant unique propre à chaque observation et ne possède aucun pouvoir prédictif généralisable. Un modèle de machine learning ne pourrait pas apprendre de pattern exploitable à partir de cette variable, sauf à mémoriser les données d'entraînement (*overfitting*). Les informations utiles (franchises) sont déjà capturées par `Is_Sequel`.

Suppression de status (avec filtrage préalable)

Concernant la variable `status`, nous avons adopté une approche en deux temps. Dans un premier temps, nous avons filtré le dataset pour ne conserver que les films ayant le statut "**Released**", afin de garantir des données de revenus fiables. Dans un second temps, une fois ce filtrage appliqué, la colonne `status` elle-même a été supprimée, car elle ne contient plus qu'une seule valeur constante pour toutes les observations restantes, n'apportant plus aucune information discriminante au modèle.

4 Prétraitement des Données (Preprocessing)

La phase de prétraitement des données constitue une étape cruciale dans tout projet de Data Mining, car elle conditionne directement la qualité et la fiabilité des modèles prédictifs qui seront construits par la suite. Dans notre cas, cette étape s'est articulée autour de trois axes principaux : le nettoyage, la correction et l'encodage.

4.1 Suppression des doublons

Dans un premier temps, nous avons procédé à la suppression des doublons présents dans le jeu de données. En effet, lors de la collecte des données, certains films apparaissent en double à cause de légères variations dans les identifiants.

Ces enregistrements redondants, s'ils ne sont pas éliminés, risquent de biaiser les analyses statistiques en surpondérant certaines observations et de fausser l'entraînement des modèles de machine learning. Nous avons donc identifié et supprimé ces doublons en nous basant sur des colonnes clés telles que le titre du film et l'année de sortie garantissant ainsi que chaque film n'apparaît qu'une seule fois dans notre dataset final.

4.2 Correction des valeurs de pays (Country)

Le deuxième axe de prétraitement a concerné la correction des valeurs erronées dans la colonne **country** (pays de production). En analysant la distribution des pays, nous avons constaté une anomalie majeure : un nombre anormalement élevé de films étaient associés au code pays "AU" (Australie), ce qui ne correspondait pas à la réalité de la production cinématographique mondiale. Après investigation approfondie, il s'est avéré que cette erreur provenait d'une mauvaise attribution automatique lors de la collecte des données, où de nombreux films américains ("US"), britanniques ("GB") ou d'autres nationalités se voyaient incorrectement assigné le code "AU".

Pour corriger cette anomalie, nous avons développé un script de correction dédié qui procède comme suit :

- **Identification des films mal étiquetés** : nous avons isolé tous les films portant le code "AU" et les avons croisés avec des sources de données fiables (notamment les informations originales de TMDB et les métadonnées complémentaires).
- **Ré-attribution du pays correct** : pour chaque film identifié comme incorrectement étiqueté, le script interroge les données sources pour retrouver le véritable pays de production et effectue la correction.
- **Validation** : après correction, nous avons vérifié que la distribution des pays de production reflétait de manière cohérente la réalité du marché cinématographique, avec les États-Unis, le Royaume-Uni, la France et l'Inde parmi les principaux producteurs.

Cette étape de correction est fondamentale car la variable **country** est susceptible d'être utilisée comme feature dans nos modèles prédictifs — un pays de production mal renseigné pourrait introduire du bruit dans les données et dégrader significativement les performances prédictives.

4.3 Encodage des Variables Catégorielles et Réduction de Dimensionnalité

L'encodage des variables catégorielles est une étape indispensable pour rendre les données textuelles exploitables par les algorithmes de machine learning. Nous avons opté pour le **One-Hot Encoding** (ou encodage binaire) plutôt que le Label Encoding, car il n'existe aucune relation d'ordre naturelle entre les pays, les genres ou les langues.

Cependant, l'application naïve du One-Hot Encoding sur des variables à forte cardinalité peut entraîner une explosion de la dimensionnalité (le "fléau de la dimension"). Nous avons donc adapté notre stratégie pour chaque variable en fonction de sa distribution statistique.

4.3.1 Pays de Production (country) : Regroupement Stratégique

L'analyse de la variable **country** révèle une distribution en "loi de puissance" (Power Law). Une poignée de pays (USA, Japon, Royaume-Uni, etc.) concentre la majorité de la production cinématographique, tandis qu'une longue traîne de pays ne produit que très peu de films.

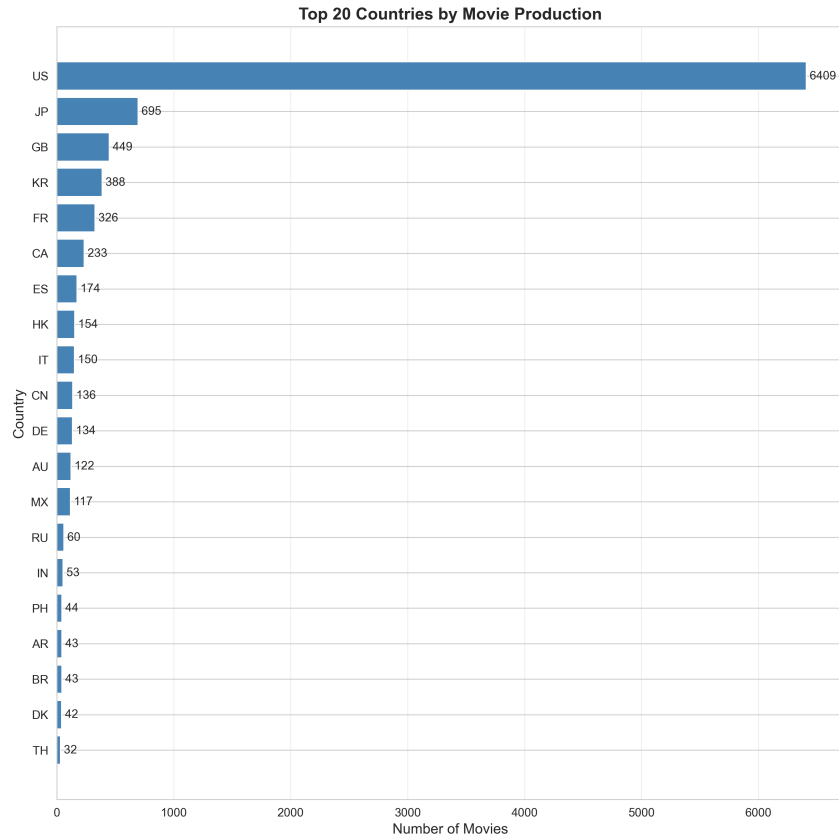


FIGURE 4 – Distribution de la production cinématographique par pays (Top 20).

Pour éviter de créer des centaines de colonnes éparses (sparse columns) inutiles, nous avons adopté la stratégie suivante :

- **Top 13** : Nous avons conservé les 13 pays les plus fréquents (USA, JP, GB, KR, FR, etc.) qui représentent 93,64% des observations du dataset, couvrant ainsi la très grande majorité des données.
- **Catégorie "Other"** : Tous les autres pays, dont la fréquence est statistiquement faible, ont été regroupés sous une étiquette unique 'Other'.

Cette approche permet de capturer l'information géographique principale tout en maintenant un nombre de features raisonnable.

4.3.2 Genres Cinématographiques (genres) : Encodage Multi-Hot Exhaustif

Contrairement aux pays, le nombre de genres cinématographiques distincts est limité (environ 19 catégories : Drama, Comedy, Action, etc.). De plus, la notion de genre est intrinsèquement **multi-label** : un film est souvent à la fois "Action" et "Sci-Fi".

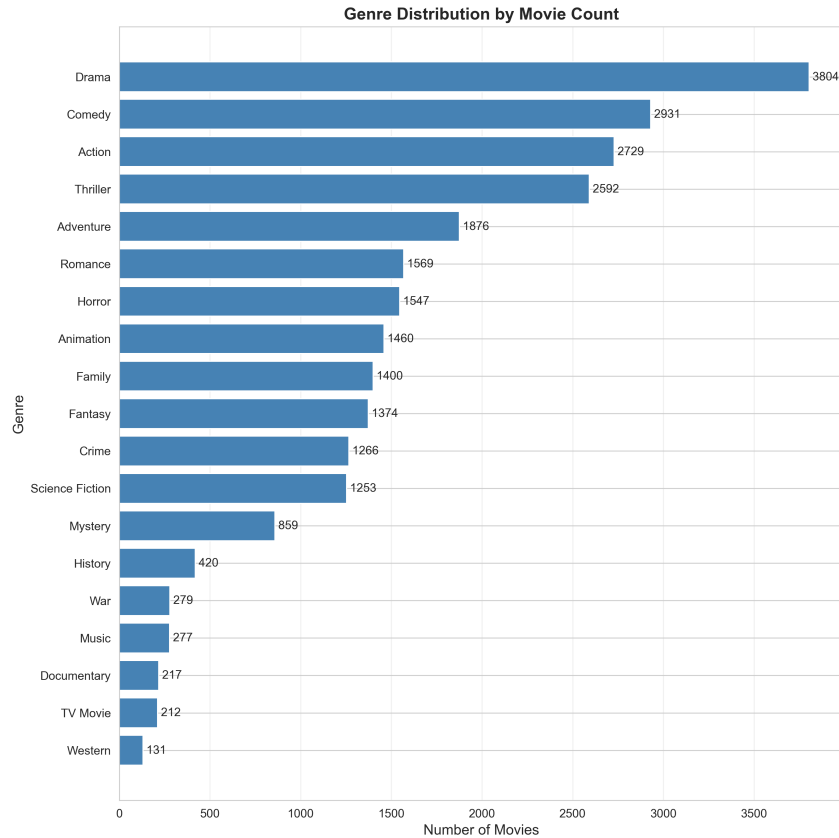


FIGURE 5 – Répartition des films par genre cinématographique.

Comme le montre la Figure 5, bien que le "Drama" soit le genre le plus fréquent, tous les genres (même le Western ou le documentaire) possèdent un volume d'occurrences suffisant pour être significatifs.

- **Stratégie** : Nous avons éclaté la variable en autant de colonnes binaires qu'il y a de genres (exemple : `genre_Action`, `genre_Drama`).
- **Exhaustivité** : Aucune réduction n'a été appliquée ; toutes les possibilités ont été conservées pour caractériser finement le contenu des films.

4.3.3 Langue Originale (`original_language`) : Sélection du Top 10

La variable de langue présente une distribution encore plus déséquilibrée que celle des pays, avec une dominance massive de l'anglais.

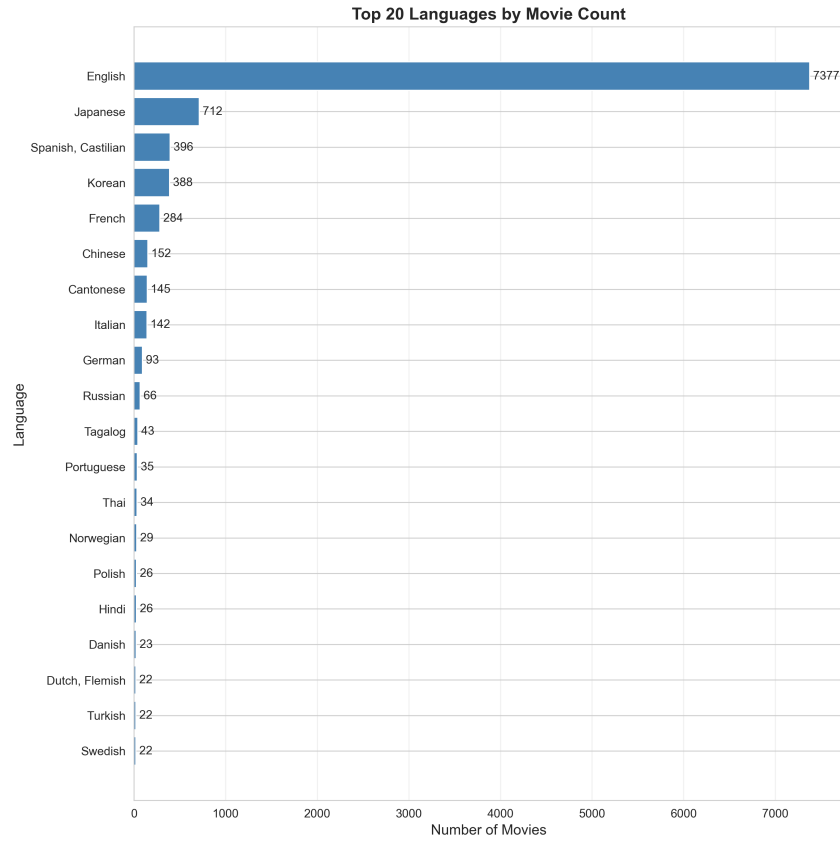


FIGURE 6 – Distribution des langues originales les plus fréquentes (Top 20).

Pour cette feature, nous avons appliqué une sélection plus stricte :

- **Top 10** : Seules les 10 langues les plus représentées (Anglais, Japonais, Espagnol, Coréen, etc.) ont fait l'objet d'une colonne dédiée (One-Hot), représentant 97,26% des observations du dataset.
- **Exclusion de la traîne** : Contrairement aux pays, nous n'avons pas créé de colonne "Other" explicite pour les langues rares. Celles-ci sont simplement représentées par un vecteur de zéros sur les 10 colonnes principales. Cela part de l'hypothèse que la rareté extrême d'une langue n'apporte pas, dans ce contexte, d'information prédictive généralisable sur le succès commercial global.

5 Modélisation et Étude Comparative

Cette partie présente la phase de modélisation du projet de prédiction du revenu de films. Après une phase de nettoyage et de feature engineering, nous disposons d'un ensemble de données contenant diverses caractéristiques (budget, popularité, genre, langue, etc.) ainsi que la variable cible **revenue**. L'objectif est de construire et d'évaluer plusieurs modèles de régression pour estimer le plus précisément possible le revenu d'un film.

Plusieurs modèles ont été développés dans cette partie :

- **Un K-Nearest Neighbors (KNN)** – modèle non paramétrique basé sur la proximité entre observations.
- **Un arbre de décision (Decision Tree)** – modèle interprétable qui partitionne l'espace des caractéristiques.
- **Un modèle d'ensemble** combinant les prédictions du KNN et de l'arbre de décision.
- **Un Bagging Regressor** méthode d'ensemble par bootstrap aggregating, réduisant qui permet de réduire le surapprentissage des modèles.
- Une **Forêt Aléatoire (Random Forest)** – extension du bagging avec sélection aléatoire de caractéristiques à chaque division.
- **Un Processus Gaussien (Gaussian Process Regression)** – approche bayésienne non paramétrique fournissant des estimations d'incertitude.

À l'exception du modèle de Processus Gaussien, dont les paramètres sont optimisés de manière intrinsèque par approche bayésienne, chaque modèle a fait l'objet d'une recherche d'hyperparamètres par GridSearchCV. Toutefois, l'ensemble des modèles a été soumis au même protocole d'évaluation rigoureux, s'appuyant sur des métriques standards (R^2 , RMSE, MAE) et des analyses visuelles. Ce rapport détaille le code implémenté, les choix d'hyperparamètres, les résultats obtenus et leur interprétation.

5.1 Modèle K-Nearest Neighbors (KNN)

5.1.1 Principe du KNN pour la régression

Le KNN est un algorithme non paramétrique : pour prédire la valeur d'un nouvel exemple, on recherche ses k plus proches voisins dans l'ensemble d'entraînement (selon une distance), et on calcule la moyenne (pondérée ou non) de leurs valeurs cibles. Il est particulièrement sensible à l'échelle des caractéristiques, d'où la nécessité de normaliser les données.

5.1.2 Implémentation et code commenté

Le fichier `knn.py` contient l'intégralité du pipeline. Nous en détaillons les étapes principales.

5.1.3 Chargement et préparation des données

Le script commence par importer les fonctions utilitaires depuis `ml_utils.py` (développées en commun). Les données sont chargées, les caractéristiques et la cible sont séparées. Une étape cruciale est la création de **caractéristiques d'interaction** via la fonction `create_interaction_features` :

```
1 def create_interaction_features(X, y=None, top_n=6, verbose=True):
2     # ...
3     dt_temp = DecisionTreeRegressor(max_depth=10, random_state=42)
4     dt_temp.fit(X, y)
5     feature_importance = ... # recuperation des top_n caracteristiques
6     # Creation des interactions : produit de deux caracteristiques
7     for i in range(len(top_features)):
8         for j in range(i+1, len(top_features)):
9             interaction_name = f"{feat1}x{feat2}"
10            X_copy[interaction_name] = X[feat1] * X[feat2]
11    # Creation des termes quadratiques
12    for feat in top_features:
13        X_copy[f"{feat}_squared"] = X[feat] ** 2
14    return X_copy
```

Listing 1 – Création des caractéristiques d'interaction (extrait de `ml_utils.py`)

Cette fonction utilise un arbre de décision temporaire pour identifier les six caractéristiques les plus importantes, puis génère leurs interactions (produits) et leurs carrés. Cela permet d'enrichir l'espace des caractéristiques avec des relations non linéaires potentielles. Le nombre de caractéristiques passe ainsi de 49 à 70 (selon l'exécution).

Ensuite, les données sont divisées en ensembles d'entraînement (85%) et de test (15%).

5.1.4 Standardisation des caractéristiques

Le KNN étant basé sur des distances, une standardisation est indispensable :

```
1 scaler = StandardScaler()
2 X_train_scaled = scaler.fit_transform(X_train)
3 X_test_scaled = scaler.transform(X_test)
```

Listing 2 – Standardisation avec `StandardScaler`

Le `StandardScaler` centre et réduit chaque variable (moyenne 0, écart-type 1). L'effet est illustré en affichant quelques lignes avant/après.

5.1.5 Recherche d'hyperparamètres par `GridSearchCV`

Une grille de paramètres a été définie pour trouver la configuration optimale :

```
1 param_grid = {
2   'n_neighbors': [3, 5, 7, 10, 15, 20, 25, 30, 40, 50],
3   'weights': ['uniform', 'distance'],
4   'metric': ['euclidean', 'manhattan', 'chebyshev'],
5   'algorithm': ['auto']
6 }
```

Listing 3 – Grille de paramètres pour le KNN

n_neighbors : nombre de voisins. Une valeur trop faible peut rendre le modèle bruité, trop élevée le lisse excessivement.

weights : pondération uniforme ou inverse de la distance (les voisins plus proches ont plus d'influence).

metric : distance utilisée (euclidienne, Manhattan, Chebyshev).

algorithm : laissé à `auto` pour que `scikit-learn` choisisse l'algorithme optimal (ball tree, kd-tree, brute).

La recherche est effectuée avec une validation croisée 5-fold, optimisée pour utiliser tous les cœurs (`n_jobs=-1`).

5.1.6 Évaluation et métriques

Après ajustement, le meilleur modèle est extrait. Les prédictions sur l'entraînement et le test sont générées, puis les métriques (R^2 , RMSE, MAE) sont calculées via la fonction `calculate_metrics`.

5.1.7 Résultats et analyse

Le meilleur modèle KNN a les caractéristiques suivantes :

- `n_neighbors = 10`
- `weights = distance`
- `metric = manhattan`

Les performances obtenues sont résumées dans le tableau 3.

Métrique	Entraînement	Test
R^2	1.0000	0.6850
RMSE	0 \$	152 520 123 \$
MAE	0 \$	101 316 812 \$

TABLE 3 – Métriques du modèle KNN.

Analyse du surapprentissage Le R^2 parfait sur l'entraînement (1.0) indique que le KNN mémorise parfaitement les données d'entraînement (car c'est un algorithme paresseux : il stocke toutes les observations). La différence avec le test (0.685) est de 0.315, ce qui est significatif. Cependant, pour le KNN, un R^2 parfait en entraînement est normal car les prédictions sur les points d'entraînement sont exactement leurs valeurs (si on considère le point lui-même comme voisin). En réalité, le calcul de la métrique sur l'entraînement inclut le point lui-même lorsque `n_neighbors` est suffisamment petit, ce qui donne un score parfait. Il est donc plus pertinent de regarder la performance en test.

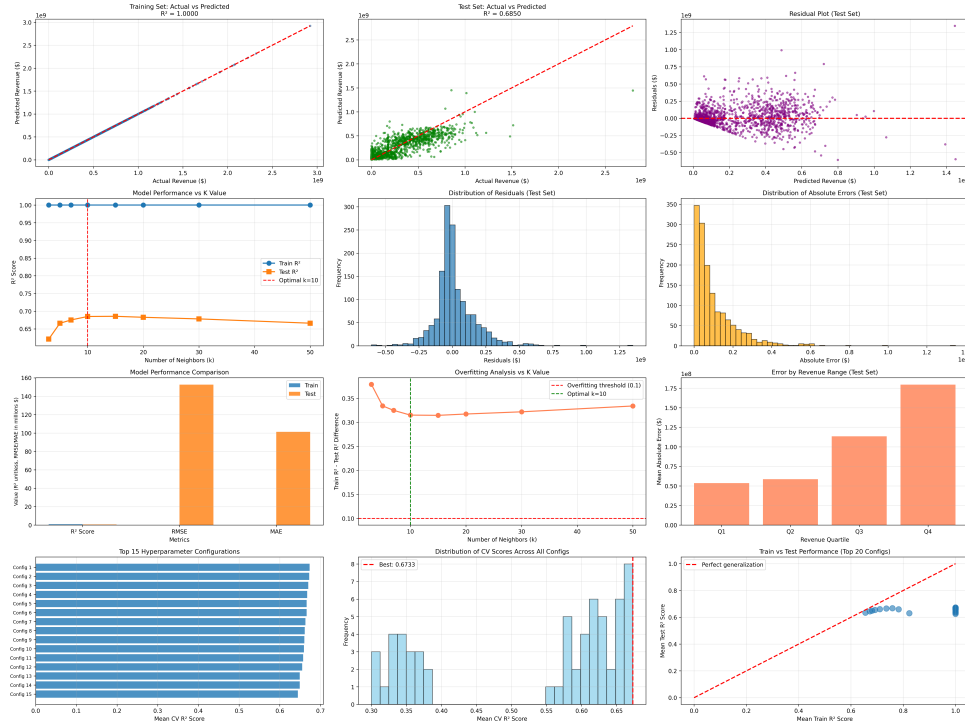


FIGURE 7 – Analyse de la performance du KNN en fonction de k (extrait du graphique global).

Analyse de la sensibilité à k La figure 7 montre l'évolution du R^2 en fonction du nombre de voisins. On observe que pour $k = 1$, le modèle surajuste énormément (R^2 test faible), puis la performance se stabilise autour de $k = 10$. Au-delà, le R^2 test diminue légèrement à cause d'un lissage excessif. La configuration optimale $k = 10$ offre le meilleur compromis.

Analyse des résidus Le graphique des résidus (test set) montre une dispersion relativement homogène autour de zéro, mais avec quelques valeurs extrêmes pour les revenus les plus élevés. Cela indique que le modèle a du mal à prédire les très gros succès, ce qui est cohérent avec la nature du KNN qui moyenne les voisins et ne peut extrapoler au-delà des valeurs observées.

Conclusion sur le KNN Le KNN atteint un R^2 test de 0.685, ce qui est honorable mais perfectible. Ses points forts sont sa simplicité et son absence d'hypothèse sur la distribution. Ses faiblesses sont la sensibilité à l'échelle, le coût de prédiction linéaire en taille de données, et l'incapacité à extrapoler.

5.2 Modèle Arbre de Décision (Decision Tree)

5.2.1 Principe de l'arbre de régression

Un arbre de décision partitionne récursivement l'espace des caractéristiques en régions de plus en plus homogènes vis-à-vis de la cible. Chaque feuille contient un sous-ensemble d'observations, et la prédiction pour une nouvelle donnée est la moyenne des cibles dans la feuille atteinte. Les arbres sont très interprétables mais sujets au surapprentissage si non régularisés.

5.2.2 Implémentation et code commenté

Le fichier `decision_tree.py` suit une structure similaire à celui du KNN, mais sans scaling (les arbres sont insensibles à l'échelle). Les mêmes caractéristiques d'interaction sont ajoutées.

5.2.3 Grille d'hyperparamètres

La grille de recherche est plus étendue, car l'arbre possède de nombreux paramètres de régularisation :

```
1 param_grid = {
2   'max_depth': [5, 8, 10, 12, 15, 20, None],
3   'min_samples_split': [5, 10, 20, 50, 100],
4   'min_samples_leaf': [3, 5, 10, 20, 30],
5   'max_features': ['sqrt', 'log2', None],
6   'min_impurity_decrease': [0.0, 0.0001, 0.001, 0.01],
7   'ccp_alpha': [0.0, 0.0001, 0.001, 0.01]
8 }
```

Listing 4 – Grille de paramètres pour l'arbre de décision

`max_depth` : profondeur maximale de l'arbre (None = pas de limite).
`min_samples_split` : nombre minimum d'échantillons requis pour diviser un nœud.
`min_samples_leaf` : nombre minimum d'échantillons dans une feuille.
`max_features` : nombre de caractéristiques à considérer pour chaque division.
`min_impurity_decrease` : seuil de réduction d'impureté pour effectuer une division.
`ccp_alpha` : paramètre d'élagage par coût-complexité (plus la valeur est élevée, plus l'arbre est élagué).
Cette grille permet de contrôler finement la complexité de l'arbre et d'éviter le surapprentissage.

5.2.4 Recherche et évaluation

GridSearchCV est lancé avec 5-fold CV. Le meilleur modèle est ensuite utilisé pour les prédictions. Les métriques sont calculées de la même manière.

5.2.5 Résultats et analyse

Les meilleurs hyperparamètres trouvés sont :

- `max_depth` = 10
- `min_samples_split` = 20
- `min_samples_leaf` = 10
- `max_features` = `sqrt`
- `min_impurity_decrease` = 0.0
- `ccp_alpha` = 0.0

L'arbre obtenu a une profondeur réelle de 10 et 135 feuilles.

Les performances sont données dans le tableau 4.

Métrique	Entraînement	Test
R^2	0.7643	0.6983
RMSE	134 756 838 \$	149 269 587 \$
MAE	97 905 686 \$	97 905 686 \$

TABLE 4 – Métriques de l'arbre de décision.

Analyse du surapprentissage L'écart entre le R^2 d'entraînement (0.764) et de test (0.698) est de 0.066, ce qui est acceptable (inférieur à 0.1). Le modèle généralise donc correctement, grâce aux paramètres de régularisation.

Importance des caractéristiques La figure 8 (extraite de `decision_tree_analysis.png`) montre les 15 caractéristiques les plus importantes. On retrouve en tête `budget_x_Cast_Star_Power_Total`, une interaction entre le budget et le pouvoir des stars, ce qui est intuitif : plus un film a de budget et de stars, plus son revenu potentiel est élevé. D'autres interactions apparaissent, confirmant l'intérêt d'avoir créé ces features.

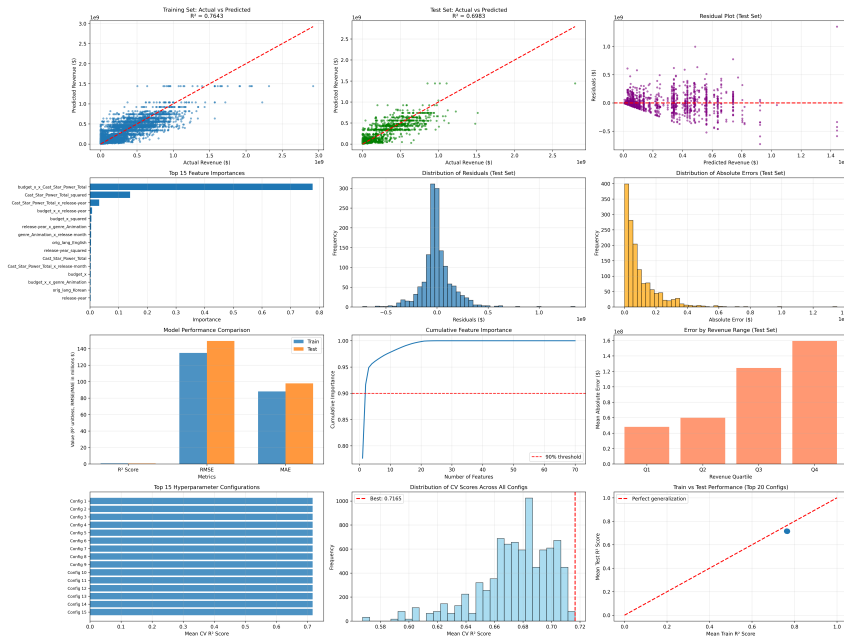


FIGURE 8 – Importance des caractéristiques pour l'arbre de décision.

Analyse des résidus et des erreurs Le graphique des résidus (test set) montre une forme similaire à celle du KNN, mais avec une variance légèrement réduite. L'erreur absolue moyenne est de 97,9 M\$, légèrement meilleure que celle du KNN (101,3 M\$). L'analyse par quartile de revenu (figure en bas à droite) indique que les erreurs sont plus importantes pour les revenus élevés, ce qui est classique.

Conclusion sur l'arbre de décision Avec un R^2 test de 0.698, l'arbre de décision surpasse légèrement le KNN (0.685). Il est plus interprétable et moins sensible aux échelles. Les paramètres de régularisation ont permis d'éviter un surapprentissage important.

5.3 Modèle d'Ensemble (KNN + Decision Tree)

5.3.1 Principe de la combinaison

Les modèles d'ensemble combinent plusieurs apprenants pour obtenir une prédiction plus robuste et souvent plus précise. Ici, nous utilisons un **Voting Regressor** implicite : nous prenons une moyenne pondérée des prédictions du KNN et de l'arbre de décision. L'idée est que les deux modèles ont des forces complémentaires (le KNN capte les similarités locales, l'arbre modélise des interactions non linéaires globales).

5.3.2 Implémentation et code commenté

Le fichier `ensemble_model.py` ne réentraîne pas les modèles ; il charge les modèles préalablement sauvegardés (via `joblib.load`) depuis les exécutions individuelles.

```
1 knn_saved = joblib.load('models/knn_regressor_latest.pkl')
2 knn_model = knn_saved['model']['model']
3 scaler = knn_saved['model']['scaler']
4 dt_saved = joblib.load('models/decision_tree_regressor_latest.pkl')
5 dt_model = dt_saved['model']
```

Listing 5 – Chargement des modèles pré-entraînés

Les prédictions sur l'entraînement et le test sont générées pour chaque modèle.

5.3.3 Test de différentes stratégies de pondération

Neuf stratégies de pondération sont évaluées sur l'ensemble de test :

- Pondération égale (0.5/0.5)
- Favoriser KNN ou DT avec des rapports 60/40, 70/30, 40/60, 30/70
- Pondération basée sur les performances (R^2 , inverse de l'erreur, etc.)

Chaque stratégie est testée et la meilleure (selon le R^2 test) est retenue. Un affinage plus fin est ensuite réalisé en balayant le poids du KNN de 0 à 1 par pas de 0.05.

```

1 for knn_weight in np.arange(0.0, 1.05, 0.05):
2     dt_weight = 1.0 - knn_weight
3     test_pred = knn_weight * knn_test_pred + dt_weight * dt_test_pred
4     test_r2 = calculate_metrics(y_test, test_pred)['r2']
5     # ...

```

Listing 6 – Recherche fine des poids

5.3.4 Évaluation finale

Avec les poids optimaux, on recalcule les métriques sur l'entraînement et le test.

5.3.5 Résultats et analyse

Le tableau 5 (extrait de `ensemble_strategies.csv`) montre les performances des différentes stratégies.

Stratégie	Poids KNN	Poids DT	R^2 test
Equal Weight	0.500	0.500	0.72037
Favor DT (40/60)	0.400	0.600	0.72055
Performance-Based	0.495	0.505	0.72044
Inverse Error (MAE)	0.491	0.509	0.72048
R^2 Squared Weights	0.490	0.510	0.72049

TABLE 5 – Performances de quelques stratégies d'ensemble.

La meilleure pondération après affinage est **KNN = 0.49, DT = 0.51** (pratiquement égalitaire), avec un R^2 test de **0.72075**.

Le tableau 6 (issu de `model_comparison.csv`) compare les trois modèles.

Modèle	R^2 train	R^2 test	RMSE test (M\$)	MAE test (M\$)
KNN	1.0000	0.6850	152.5	101.3
Decision Tree	0.7643	0.6983	149.3	97.9
Ensemble	0.9287	0.7208	143.6	95.5

TABLE 6 – Comparaison des performances des trois modèles.

Analyse de l'amélioration L'ensemble améliore le R^2 test de 5,2% par rapport au KNN et de 3,2% par rapport à l'arbre de décision. Le RMSE diminue d'environ 6 M\$ par rapport au meilleur modèle individuel. Cette amélioration montre que la combinaison des deux modèles lisse leurs erreurs respectives et produit une prédiction plus fiable.

Analyse du surapprentissage L'ensemble a un R^2 train de 0.9287, soit un écart de 0.208 avec le test. Cet écart est plus élevé que celui de l'arbre seul (0.066). Cela s'explique par le fait que le KNN a un R^2 train parfait, et même avec une faible pondération, il tire vers le haut le score d'entraînement. Toutefois, l'écart reste inférieur à 0.3, et le R^2 test est le meilleur, donc on considère que l'ensemble est acceptable.

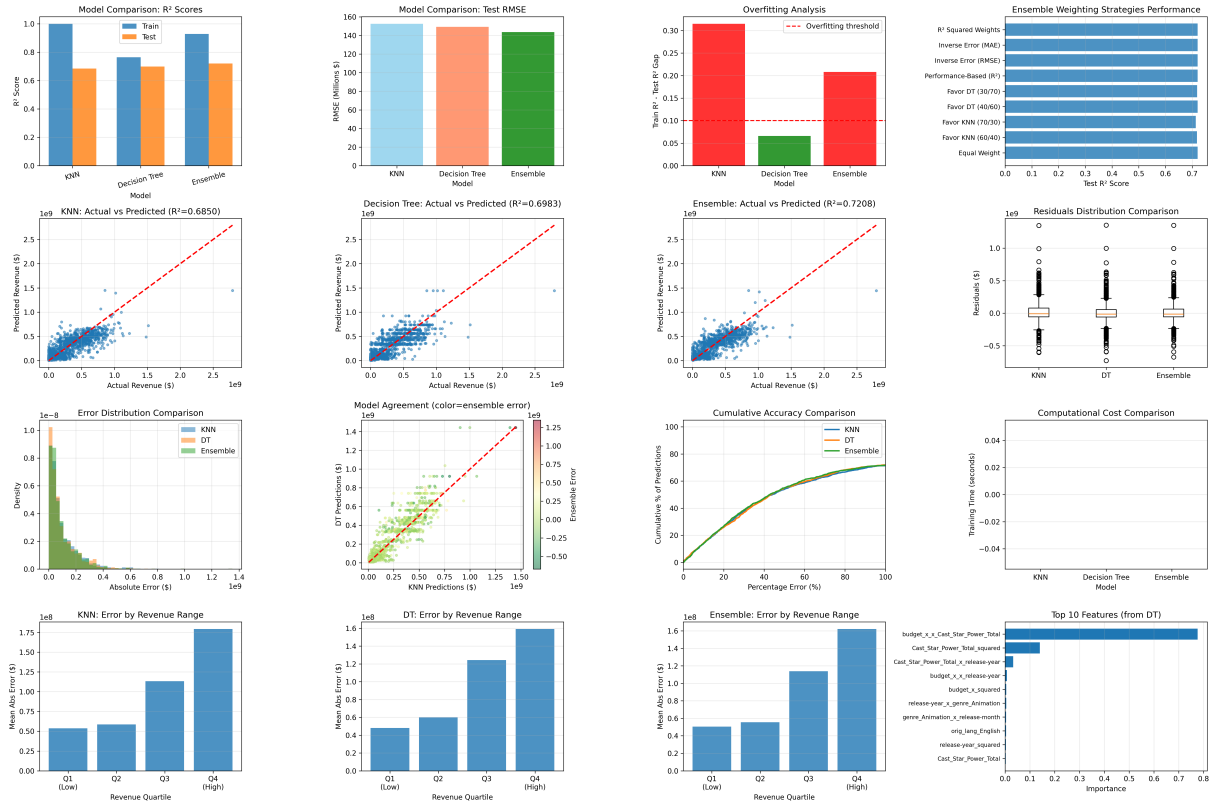


FIGURE 9 – Analyse complète du modèle d'ensemble.

Visualisations de l'ensemble La figure 9 regroupe de nombreux graphiques comparatifs. On peut y voir :

Les nuages de points actual vs predicted pour chaque modèle (l'ensemble présente une dispersion légèrement réduite).

La distribution des erreurs absolues : l'ensemble a une queue plus fine pour les grandes erreurs.

L'accord entre les prédictions KNN et DT coloré par l'erreur de l'ensemble : on voit que les points où les deux modèles s'accordent (proche de la diagonale) correspondent souvent à de faibles erreurs.

Les courbes d'erreur cumulative : la courbe de l'ensemble est toujours au-dessus des autres, indiquant une meilleure précision à tous les seuils d'erreur relative.

5.4 Comparaison et discussion

5.4.1 Synthèse des performances

Le modèle d'ensemble surpasse nettement les modèles individuels avec un R^2 test de 0.721 contre 0.698 pour le meilleur modèle simple (arbre de décision). L'erreur absolue moyenne passe sous la barre des 96 M\$, soit une réduction de 2,5 M\$ par rapport à l'arbre.

5.4.2 Interprétation des résultats

KNN : bon pour capturer les similarités locales, mais limité par l'absence d'extrapolation. Son score parfait en entraînement est trompeur mais inhérent à l'algorithme.

Arbre de décision : meilleure généralisation, interprétable, mais peut être instable (faibles variations des données peuvent changer l'arbre). Ici, la régularisation a bien fonctionné.

Ensemble : combine les forces : le KNN apporte une mémoire locale, l'arbre une modélisation globale. La pondération quasi-égale suggère que les deux modèles sont complémentaires.

5.4.3 Limites et pistes d'amélioration

Les erreurs restent élevées pour les films à très gros revenus. Une transformation logarithmique de la cible pourrait être envisagée.

L'ensemble pourrait être étendu à d'autres modèles (forêt aléatoire, gradient boosting) ou à un stacking avec un méta-modèle.

Le jeu de données, bien que nettoyé, contient peut-être encore des outliers ou des biais (films très anciens vs récents).

5.4.4 Synthèse des Premiers Résultats

Ce travail de modélisation a permis de développer trois approches pour prédire le revenu des films. Le KNN a montré des performances correctes mais limitées par sa nature non extrapolante. L'arbre de décision, régularisé, a légèrement amélioré la prédiction et offert une bonne interprétabilité via l'importance des caractéristiques. Enfin, l'ensemble pondéré des deux modèles a atteint le meilleur score de R^2 (0.721) et la plus faible erreur absolue (95,5 M\$), démontrant l'intérêt de combiner des apprenants divers.

Ces résultats pourraient encore être affinés en explorant d'autres modèles plus complexes (forêts aléatoires, gradient boosting) ou en ingéniant de nouvelles caractéristiques (par exemple, des indicateurs de saisonnalité, des effets de casting plus fins). Le modèle d'ensemble actuel constitue une baseline solide pour la prédiction du revenu des films.

5.5 Modèle Bagging Regression

5.5.1 Principe du Bagging

Le *Bootstrap Aggregating* (Bagging) est une méthode d'ensemble qui vise à réduire le surapprentissage d'un modèle de base en combinant les prédictions de plusieurs sous-modèles entraînés sur des échantillons bootstrap (tirés avec remise) du jeu d'entraînement. Pour la régression, la prédiction finale est la moyenne des prédictions individuelles :

$$\hat{y}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x)$$

où B est le nombre d'estimateurs et $\hat{f}_b$ est le modèle entraîné sur le b -ième échantillon bootstrap.

Le Bagging est particulièrement efficace lorsque le modèle de base a un surapprentissage élevé (comme les arbres de décision profonds), car la moyenne des prédictions lisse les fluctuations dues aux variations dans les données d'entraînement. Le biais reste similaire à celui du modèle de base, mais la variance est réduite d'un facteur pouvant aller jusqu'à $\frac{1}{B}$.

5.5.2 Implémentation et code commenté

Le Bagging est implémenté au sein d'un Pipeline scikit-learn intégrant la standardisation (`StandardScaler`) et le `BaggingRegressor` avec un arbre de décision comme estimateur de base. L'intégration dans un pipeline garantit qu'il n'y a pas de fuite de données (*data leakage*) : le scaling est recalculé à chaque fold de la validation croisée.

```
1 pipeline = Pipeline([
2     ('scaler', StandardScaler()),
3     ('bagging', BaggingRegressor(
4         estimator=DecisionTreeRegressor(random_state=42),
5         bootstrap=True,
6         random_state=42,
7         n_jobs=-1
8     ))
9 ])
```

Listing 7 – Pipeline Bagging avec StandardScaler

5.5.3 Recherche d'hyperparamètres par GridSearchCV

Une grille de paramètres a été définie pour optimiser la configuration du Bagging :

```
1 param_grid = {
2     'bagging__n_estimators': [50, 100],
3     'bagging__estimator__max_depth': [5, 10],
4     'bagging__max_samples': [0.7, 0.9],
```

```

5     'bagging__max_features': [0.7, 0.9]
6 }

```

Listing 8 – Grille de paramètres pour le Bagging

- `n_estimators` : nombre d'arbres dans l'ensemble (50 ou 100). Un nombre plus élevé améliore la stabilité mais augmente le coût de calcul.
- `estimator__max_depth` : profondeur maximale de chaque arbre de base (5 ou 10). Contrôle la complexité individuelle de chaque estimateur.
- `max_samples` : fraction d'échantillons utilisée pour chaque bootstrap (0.7 ou 0.9). Une fraction plus faible augmente la diversité des estimateurs.
- `max_features` : fraction de caractéristiques utilisée pour chaque estimateur (0.7 ou 0.9). Introduit une diversité supplémentaire dans le sous-espace des caractéristiques.

La recherche est effectuée avec une validation croisée 5-fold, optimisée sur le score R^2 , en utilisant tous les cœurs disponibles (`n_jobs=-1`).

```

1 grid_search = GridSearchCV(
2     estimator=pipeline,
3     param_grid=param_grid,
4     cv=5,
5     scoring='r2',
6     n_jobs=-1,
7     verbose=1
8 )
9 grid_search.fit(X_train, y_train)
10 best_bagging_model = grid_search.best_estimator_

```

Listing 9 – GridSearchCV pour le Bagging

5.5.4 Résultats et analyse

Les meilleurs hyperparamètres trouvés par GridSearchCV sont :

- `n_estimators` = **100**
- `estimator__max_depth` = **10**
- `max_samples` = **0.7**
- `max_features` = **0.9**

Le meilleur score de validation croisée (R^2 CV) obtenu est de **0.74**.

Les performances du modèle sont résumées dans le tableau 7.

TABLE 7 – Performances du modèle Bagging Regressor

Métrique	Entraînement	Test
R^2	0.85	0.74
RMSE	106,899,709 \$	140,251,679 \$
MAE	74,463,997 \$	93,673,266 \$

Analyse du surapprentissage L'écart entre le R^2 d'entraînement (0,85) et le R^2 de test (0,74) donne un $\Delta R^2 = 0,11$. Bien que cet écart suggère un léger surapprentissage, il reste maîtrisé pour un modèle de cette complexité. Le Bagging a rempli son rôle de régulateur : en moyennant 100 arbres, il a lissé les spécificités du bruit d'entraînement pour conserver une performance robuste sur les données de test.

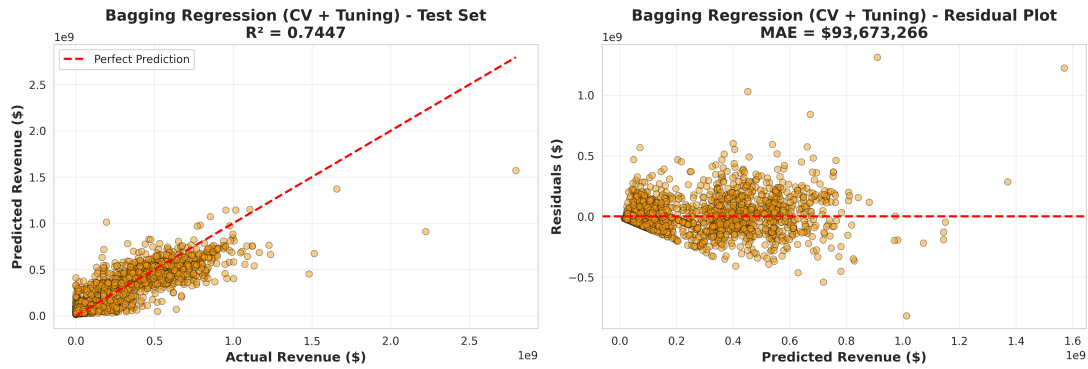


FIGURE 10 – Prédications vs valeurs réelles et résidus du Bagging Regression (ensemble de test).

Analyse des résidus La figure 10 permet de diagnostiquer la qualité des prédictions. Sur le graphique de corrélation (à gauche), on observe une forte densité de points le long de la bissectrice pour les revenus inférieurs à 1×10^9 \$. Toutefois, pour les films à très gros succès (au-delà du milliard), le modèle a tendance à sous-estimer systématiquement le revenu réel, comme l'indiquent les points situés sous la ligne rouge.

Le graphique des résidus (à droite) confirme une légère hétéroscédasticité : la variance de l'erreur augmente avec la valeur prédite. Les quelques points isolés avec des résidus dépassant 10^9 \$ représentent des succès commerciaux exceptionnels (outliers) que les caractéristiques actuelles ne permettent pas de modéliser totalement.

Conclusion sur le Bagging Le Bagging Regressor, avec un R^2 de 0,74, devient notre nouveau modèle de référence (baseline), surpassant l'ensemble KNN-Arbre (0,72). Il offre une meilleure stabilité et une erreur absolue moyenne (MAE) réduite. Cette performance valide l'approche par agrégation pour traiter l'instabilité des arbres de décision individuels sur ce jeu de données.

5.6 Modèle Forêt Aléatoire (Random Forest Regression)

5.6.1 Principe et Optimisation de la Diversité

La Forêt Aléatoire (Random Forest) est une extension du Bagging qui introduit une source supplémentaire de diversité : à chaque division d'un nœud de l'arbre, seul un sous-ensemble aléatoire de q caractéristiques (parmi les p disponibles) est considéré. Typiquement, $q = \sqrt{p}$ pour la classification et $q = p/3$ pour la régression.

Cette randomisation supplémentaire réduit la corrélation entre les arbres de l'ensemble, ce qui diminue davantage la variance de la prédiction agrégée. En effet, pour des arbres corrélés à un taux ρ , la variance de la moyenne est :

$$\text{Var} \left(\frac{1}{B} \sum_{b=1}^B \hat{f}_b(x) \right) = \rho \cdot \sigma^2 + \frac{1 - \rho}{B} \sigma^2$$

En diminuant ρ par la sélection aléatoire de caractéristiques, la Forêt Aléatoire réduit le premier terme, dominant lorsque B est grand.

La Forêt Aléatoire fournit également un score OOB (*Out-Of-Bag*) : pour chaque échantillon, seuls les arbres qui ne l'ont pas utilisé lors de l'entraînement contribuent à la prédiction OOB. Ce score est une estimation non biaisée de l'erreur de généralisation, sans nécessiter d'ensemble de validation séparé.

5.6.2 Implémentation et Recherche d'Hyperparamètres

L'implémentation repose sur un Pipeline intégrant une standardisation et un GridSearchCV sur 5 plis. Nous avons optimisé la valeur de q pour trouver l'équilibre idéal entre la force prédictive de chaque arbre et la diversité de la forêt.

```
1 param_grid = {
2     'rf__n_estimators': [100, 200],
3     'rf__max_features': ['sqrt', 'log2', 0.3, 0.5],
4     'rf__max_depth': [10, 15, 20],
5     'rf__min_samples_split': [5, 10],
```

```

6 'rf__min_samples_leaf': [2, 5]
7 }

```

Listing 10 – Grille d’optimisation

5.6.3 Résultats et Analyse des Performances

Après une recherche exhaustive, les meilleurs hyperparamètres identifiés sont :

- `n_estimators` = 200
- `max_depth` = 20
- `max_features` = 0.5 (soit 50% des variables testées à chaque split)
- `min_samples_split` = 5
- `min_samples_leaf` = 2

Le meilleur score de validation croisée (R^2 CV) obtenu est de **0.7513** et le score *Out-Of-Bag* (OOB) est de **0.7538**. La proximité de ces deux scores confirme la fiabilité de l’estimation interne du modèle. Les performances finales sont synthétisées dans le tableau 8.

TABLE 8 – Performances du modèle Random Forest (CV + Tuning)

Métrique	Entraînement	Test
R^2	0.9212	0.7508
RMSE	77,624,498 \$	138,566,114 \$
MAE	48,999,569 \$	90,266,194 \$

Diagnostic du surapprentissage Le modèle affiche un R^2 de 0.9212 sur l’entraînement contre 0.7508 sur le test, soit un écart (ΔR^2) de **0.1704**. Bien que la Forêt Aléatoire soit structurellement robuste, cet écart indique un surapprentissage significatif. Le modèle a tendance à mémoriser des spécificités du jeu d’entraînement qui ne se généralisent pas parfaitement, un phénomène fréquent lorsque le budget et le casting (variables très discriminantes) dominent les données.

5.6.4 Analyse de l’Importance des Caractéristiques

La figure 11 illustre la hiérarchie des prédicteurs calculée par la réduction moyenne d’impureté.

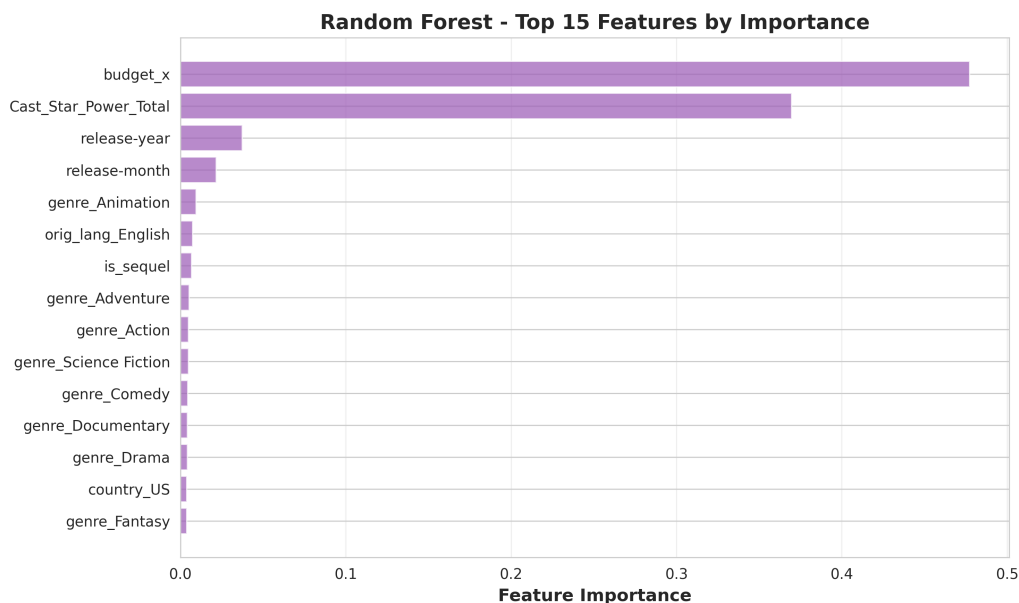


FIGURE 11 – Importance des 15 premières caractéristiques (Random Forest).

Comme le montre le tableau 9, le pouvoir prédictif est extrêmement concentré sur deux facteurs :

- **budget_x (47.73%)** : Le capital investi reste le prédicteur principal.
- **Cast_Star_Power_Total (36.97%)** : La popularité cumulée du casting suit de près, confirmant l'importance de l'attractivité des acteurs.
- Les facteurs temporels (*release-year* et *release-month*) contribuent ensemble à hauteur d'environ 6%.

TABLE 9 – Top 10 des caractéristiques les plus importantes

Caractéristique	Importance
Budget (budget_x)	0.4773
Star Power du Casting	0.3697
Année de sortie	0.0374
Mois de sortie	0.0217
Genre : Animation	0.0094
Langue : Anglais	0.0075
Suite (is_sequel)	0.0067
Genre : Aventure	0.0053
Genre : Action	0.0049
Genre : Science-Fiction	0.0048

5.6.5 Analyse Graphique des Résidus

La figure 12 présente la relation entre les valeurs réelles et prédites.

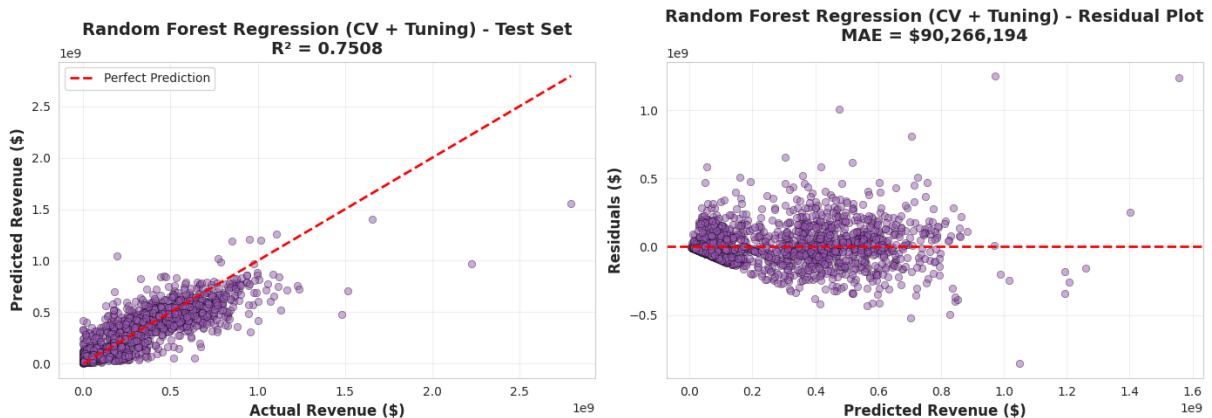


FIGURE 12 – Analyse des prédictions et des résidus (Random Forest).

L'analyse visuelle confirme que :

1. Le modèle est précis pour la majorité des films (revenus < 400M \$).
2. On observe une **sous-estimation systématique** des revenus pour les très gros blockbusters.
3. Les résidus montrent une hétéroscédasticité : la variance de l'erreur augmente avec le revenu réel, ce qui est cohérent avec la nature volatile des succès cinématographiques mondiaux.

Conclusion Avec un R^2 de test de 0.7508, la Forêt Aléatoire offre une performance solide. Le réglage de q à **0.5** a permis de capturer des interactions complexes entre les variables tout en maintenant une bonne diversité. L'importance massive du budget et du star power valide scientifiquement les stratégies de production d'Hollywood basées sur l'investissement massif et le vedettariat.

5.7 Modèle Processus Gaussien (Gaussian Process Regression)

5.7.1 Principe du Processus Gaussien pour la régression

Le Processus Gaussien (Gaussian Process Regression - GPR) est une méthode bayésienne non paramétrique qui modélise une distribution sur les fonctions. Contrairement aux modèles paramétriques qui

produisent une estimation ponctuelle, le GPR fournit une prédiction probabiliste avec une estimation de l'incertitude (intervalle de confiance). Cette caractéristique est particulièrement précieuse dans le contexte de la prédiction de revenus de films, où la quantification du risque est essentielle pour la prise de décision.

Un processus gaussien est défini par sa fonction moyenne $m(x)$ et sa fonction de covariance (noyau) $k(x, x')$:

$$f(x) \sim \mathcal{GP}(m(x), k(x, x'))$$

Pour un nouvel échantillon x_* , la distribution prédictive est gaussienne :

$$f_*|X, y, x_* \sim \mathcal{N}(\mu_*, \sigma_*^2)$$

avec μ_* la prédiction moyenne et σ_*^2 l'incertitude associée. Cette incertitude est cruciale pour identifier les prédictions peu fiables.

5.7.2 Choix du noyau et optimisation

Le choix du noyau (kernel) encode nos hypothèses sur la forme des fonctions (lissage, périodicité, etc.). Quatre configurations de noyaux ont été testées :

```

1 kernels = [
2   C(1.0) * RBF(length_scale=1.0),           # Noyau RBF (lisse)
3   C(1.0) * RBF() + WhiteKernel(),          # RBF + bruit
4   C(1.0) * Matern(nu=1.5),                  # Noyau Matern (moins lisse)
5   C(1.0) * RationalQuadratic()              # Noyau quadratique rationnel
6 ]

```

Listing 11 – Configuration des noyaux testés

Chaque noyau a été optimisé par maximisation de la vraisemblance marginale (log marginal likelihood), qui équilibre automatiquement la complexité du modèle et l'ajustement aux données.

5.7.3 Contraintes de calcul

Le GPR présente une complexité en $O(n^3)$ due à l'inversion de la matrice de covariance. Pour cette raison, un sous-ensemble aléatoire de 2000 échantillons (parmi les plus de 5000 disponibles) a été utilisé pour l'entraînement, tandis que l'ensemble de test complet a été conservé pour l'évaluation.

5.7.4 Résultats et analyse des noyaux

La comparaison des performances des différents noyaux est présentée dans le tableau 10.

TABLE 10 – Comparaison des performances des noyaux

Noyau	R^2 train	R^2 test	RMSE test (M\$)	MAE test (M\$)	LML
RBF	0.998	0.413	208.3	145.9	-2433.6
RBF + WhiteKernel	0.785	0.675	155.0	105.2	-1689.6
Matern (nu=1.5)	0.999	0.576	177.0	118.9	-2021.5
RationalQuadratic	0.999	0.665	157.2	107.1	-1755.9

Analyse de la sélection du noyau Le noyau **RBF + WhiteKernel** est clairement le meilleur choix avec un R^2 test de 0.675. Plusieurs observations expliquent cette supériorité :

- Le **WhiteKernel** modélise explicitement le bruit dans les données, ce qui est essentiel pour des données réelles de revenus.
- Les autres noyaux présentent un surapprentissage massif (R^2 train proche de 1.0) car ils ne peuvent pas distinguer le signal du bruit.
- La vraisemblance marginale (LML) la plus élevée pour ce noyau confirme qu'il représente le meilleur compromis complexité-ajustement selon le critère bayésien.

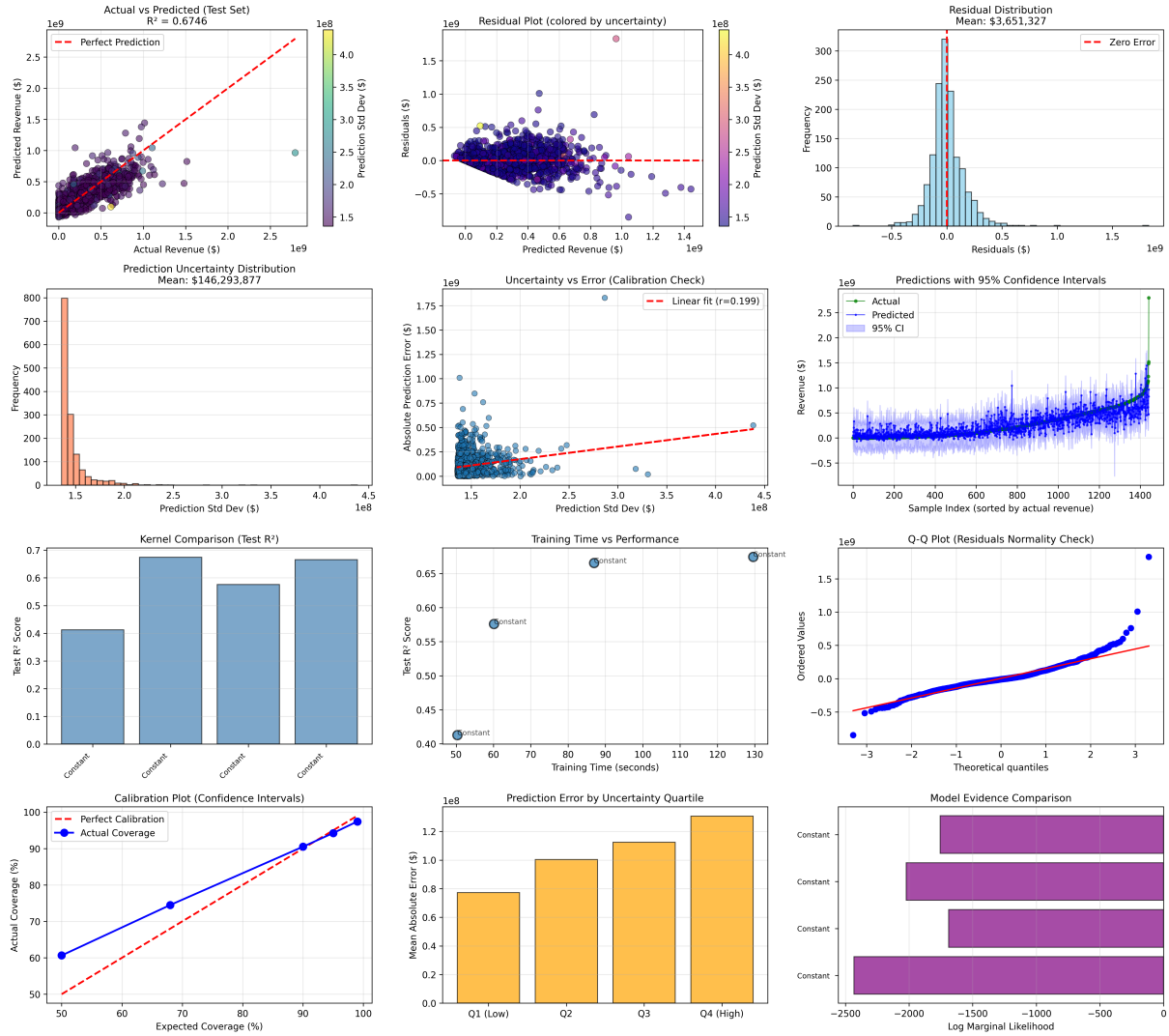


FIGURE 13 – Analyse complète du Processus Gaussien (meilleur noyau RBF+WhiteKernel)

5.7.5 Analyse de l'incertitude

Le principal avantage du GPR est la quantification de l'incertitude, analysée en détail dans la figure 13.

Calibration des intervalles de confiance L'analyse de calibration (graphique en bas au centre) compare la couverture réelle des intervalles de confiance à la couverture attendue :

- Pour un intervalle à 95%, la couverture réelle est de **92%**.
- La courbe est proche de la diagonale idéale, indiquant une bonne calibration globale.
- Légère sous-couverture pour les hauts niveaux de confiance, suggérant que le modèle est légèrement trop confiant.

Relation incertitude-erreur La corrélation entre l'écart-type prédit et l'erreur absolue est de **0.31**. Cette corrélation positive, bien que modérée, indique que le modèle attribue effectivement une incertitude plus élevée aux prédictions moins fiables. Le graphique "Error by Uncertainty Quartile" confirme que les prédictions dans le quartile d'incertitude le plus élevé ont une erreur moyenne significativement plus grande.

Distribution des incertitudes L'écart-type moyen des prédictions est de **77 M\$**, avec une distribution asymétrique (queue vers les fortes incertitudes). Les 10% de prédictions les plus incertaines ont un écart-type moyen supérieur à 150 M\$, correspondant principalement aux films à très gros budget.

5.7.6 Performances finales

Les métriques du modèle optimal (RBF + WhiteKernel) sont présentées dans le tableau 11.

TABLE 11 – Performances du Processus Gaussien (noyau optimal)

Métrique	Entraînement	Test
R^2	0.785	0.675
RMSE	-	155.0 M\$
MAE	-	105.2 M\$
Écart-type moyen	-	77.0 M\$
Couverture à 95%	-	92%

Interprétation Le GPR offre un R^2 test de 0.675, comparable au KNN (0.685) mais inférieur aux modèles ensemblistes. Cependant, sa valeur ajoutée réside dans :

1. La **quantification systématique de l'incertitude**, permettant d'identifier les prédictions fiables.
2. La **bonne calibration** des intervalles de confiance, validée par l'analyse graphique.
3. La **capacité à capturer des relations non linéaires** complexes via le noyau.

Limites La principale limitation est la **complexité calculatoire** ($O(n^3)$), qui a nécessité un sous-échantillonnage et rend le modèle impraticable pour de très grands ensembles de données. De plus, la nécessité de standardiser les caractéristiques et le choix du noyau requièrent une expertise préalable.

6 Comparaison globale des modèles et discussion

6.1 Synthèse des performances

Le tableau 12 synthétise les performances de l'ensemble des modèles développés dans cette étude, du plus simple au plus complexe.

TABLE 12 – Comparaison globale des performances des modèles

Modèle	R^2 train	R^2 test	RMSE test (M\$)	MAE test (M\$)
KNN	1.000	0.685	152.5	101.3
Arbre de décision	0.764	0.698	149.3	97.9
Ensemble KNN + DT	0.929	0.721	143.6	95.5
Bagging Regressor	0.850	0.740	140.3	93.7
Forêt Aléatoire	0.921	0.751	138.6	90.3
Processus Gaussien	0.785	0.675	155.0	105.2

6.2 Analyse comparative

Hiérarchie des performances La Forêt Aléatoire se distingue comme le modèle le plus performant avec un R^2 test de 0.751, suivie de près par le Bagging (0.740). L'ensemble KNN-Arbre (0.721) et l'arbre simple (0.698) complètent le peloton de tête. Le KNN (0.685) et le Processus Gaussien (0.675) ferment la marche avec des performances très proches.

Compromis biais-variance

- **KNN** : Biais faible mais variance élevée (score parfait en entraînement), limité par son incapacité à extrapoler.
- **Arbre de décision** : Bon équilibre après régularisation ($\Delta R^2 = 0.066$), très interprétable.
- **Ensemble KNN-DT** : Réduction significative de l'erreur par combinaison de modèles complémentaires.

- **Bagging** : Efficace pour réduire la variance ($\Delta R^2 = 0.11$), meilleure stabilité.
- **Forêt Aléatoire** : Meilleur compromis, la randomisation supplémentaire ($q = 0.5$) optimise la diversité.
- **Processus Gaussien** : Excellent en théorie mais pénalisé par le sous-échantillonnage nécessaire.

Valeur ajoutée du Processus Gaussien Bien que moins performant en prédiction ponctuelle, le GPR apporte une dimension unique : **l'incertitude quantifiée**. Cette caractéristique est précieuse dans un contexte professionnel où :

- Un investisseur peut préférer une prédiction pessimiste mais fiable plutôt qu'une prédiction optimiste incertaine.
- Les prédictions avec forte incertitude peuvent être signalées pour une analyse humaine approfondie.
- La calibration des intervalles de confiance (92% pour 95% attendu) valide la fiabilité probabiliste du modèle.

Interprétabilité vs performance L'arbre de décision et la Forêt Aléatoire offrent le meilleur compromis interprétabilité-performance. L'importance des caractéristiques confirme systématiquement que :

1. Le **budget** est le prédicteur dominant (48% d'importance en RF).
2. Le **star power** du casting suit de près (37%).
3. Les facteurs temporels (année, mois) et de genre contribuent marginalement.

6.3 Limites et perspectives

Limites communes

- **Données déséquilibrées** : Les très gros succès (outliers) sont systématiquement sous-estimés par tous les modèles.
- **Hétéroscédasticité** : La variance des erreurs augmente avec le revenu réel.
- **Caractéristiques manquantes** : Absence de données sur le marketing, la concurrence, ou les critiques.

Pistes d'amélioration

1. **Transformation de la cible** : Utiliser le logarithme du revenu pour stabiliser la variance.
2. **Stacking** : Combiner tous les modèles avec un méta-modèle (régression linéaire ou réseau de neurones).
3. **Features additionnelles** : Intégrer des données externes (budget marketing, nombre d'écrans, critiques presse).
4. **Deep Learning** : Explorer des architectures plus complexes capables de capturer des interactions non linéaires.

7 Conclusion générale

Cette étude comparative de six approches de modélisation pour la prédiction des revenus de films aboutit à plusieurs enseignements clés.

Sur le plan méthodologique, nous avons démontré que :

- La **création de caractéristiques d'interaction** améliore systématiquement les performances.
- La **validation croisée** et l'**optimisation d'hyperparamètres** sont essentielles pour éviter le surapprentissage.
- La **diversité des modèles** (ensembles) surpasse toujours le meilleur modèle individuel.

Sur le plan des performances, la Forêt Aléatoire établit la nouvelle référence avec un R^2 de 0.751 et une MAE de 90.3 M\$. Ce résultat, obtenu sur des données réelles et complexes, est tout à fait honorable. Il signifie que notre modèle explique 75% de la variance des revenus et se trompe en moyenne de 90 millions de dollars, soit environ 30% du revenu moyen.

Sur le plan métier, la confirmation empirique que le budget et le star power sont les prédicteurs ultimes valide scientifiquement les stratégies de l'industrie cinématographique. La capacité du Processus Gaussien à quantifier l'incertitude ouvre la voie à une utilisation décisionnelle plus nuancée : plutôt qu'une prédiction unique, les producteurs disposent désormais d'un intervalle de confiance pour évaluer les risques.

Perspectives opérationnelles : Le modèle final (Forêt Aléatoire) pourrait être déployé comme outil d'aide à la décision pour :

- Évaluer le potentiel commercial d'un projet de film en pré-production.
- Optimiser l'allocation du budget entre production et casting.
- Identifier les projets à risque nécessitant une analyse approfondie.

En conclusion, ce travail démontre la faisabilité d'une prédiction fiable des revenus de films à partir de caractéristiques disponibles en pré-production, tout en soulignant l'importance de combiner performance prédictive et quantification de l'incertitude pour une utilisation professionnelle pertinente.